Zurich ServiceNow AI Platform Capabilities

Last updated: January 16, 2026

Some examples and graphics depicted herein are provided for illustration only. No real association or connection to ServiceNow products or services is intended or should be inferred.

This PDF was created from content on docs.servicenow.com. The web site is updated frequently. For the most current ServiceNow product documentation, go to docs.servicenow.com.

Company Headquarters
2225 Lawson Lane
Santa Clara, CA 95054
United States
(408)501-8550

CMDB CI Lifecycle Management (legacy)

From the time of its creation to the time that it is no longer needed, a CMDB CI would typically transition through several operational states while undergoing various operations. CI LIfecycle Management provides the mechanism to define states and actions for a CI and lets you apply appropriate actions based on a CI's state to tailor the management of CI lifecycle to business needs.

The CMDB Data Manager is now a more comprehensive and integrated solution for managing CI life cycle operations such as deletion and archival, in bulk. For information about the CMDB Data Manager, see Working with CMDB Data Manager.

Terms associated with CI Lifecycle Management:

Operational states

A set of states that a CI can be at such as 'Operational' or 'Repair in Progress'. A CI can be associated with only a single operational state at any given time. The choices for operational states are based on the operational_status field in the [cmdb_ci] table. There are several operational states that are defined in the base system such as 'Retired' and 'Repair in Progress'. You can modify this list to reflect operational states that are relevant in your business.

> **Note:** By default, Service Mapping is configured to ignore all host CIs for which the value of Operational status [operational_status] is not **1** (Operational) or the value of status [install_status] is **100** (absent). For additional information about this behavior, see Preparing customized ServiceNow deployments to work with Service Mapping [KB0647574] in the HI Knowledge Base.

CI Lifecycle Management allows multiple operators and automations to simultaneously set different operational states of a CI. Since a CI cannot be associated with multiple operational states, it is important to configure each operational state with a priority. These priorities are then used in such situation to determine which of the operational states is the cumulative operational state.

CI actions

A set of actions that can be applied to a CI during its lifetime. You can define CI actions that are relevant in your business.

Compatible CI Actions

CI Lifecycle Management allows a CI to have multiple active CI actions simultaneously, however they must be specifically defined as compatible. By default, there are no two actions for a CI that are compatible with each other. You can change this behavior by specifying pairs of actions that are compatible and thus allowed to be applied simultaneously to a CI. For example, you can specify that the 'Patching' and the 'Provisioning' CI actions are compatible making it possible to apply both simultaneously to a CI.

Not Allowed CI Actions

By default, any CI action can be applied to any CI. You can restrict this behavior by defining a rule that an action is not allowed for a CI when it is in a specific operational state. For example, you can define a Not Allowed CI Action in which it is not allowed to apply the 'Provisioning' action to a Linux Server that is in a 'Non-Operational' state.

Not Allowed Operational Transitions

By default, transitions are allowed from any operational state to another. You can restrict this behavior by defining a rule that for a specified CI, a transition from a certain operational state to another operational state is not allowed. For example, you can define that for a Linux Server it is not allowed to transition from 'Repair in progress' to 'Non-Operational'.

Requestor

A requestor can be a workflow or a non-workflow operator that is trying to set operational states and apply CI actions. Each requestor has an associated requestor ID that is a GUID and that can be an active workflow context or a non-workflow registered operator ID.

Lease time

A time period that each requestor (especially non-workflow operators) can provide, during which a specified CI action is allowed to be active for a specified CI.

CMDB CI Lifecycle Management provides a set of APIs to manage CI operational states and CI actions. And the UI where you define a set of rules to restrict certain operational state transitions and to restrict actions based on operational states. It also provides a mechanism to audit CI operational state and CI actions during the entire CI lifecycle.

Providers such as automation, workflows, or Change Management can use CI Lifecycle Management as a mechanism to manage CI operational states and apply CI actions. By default, the behavior of CI Lifecycle Management has no restrictions on some operations, and full restrictions on other operations. The CI Lifecycle Management UI lets you modify this default behavior by specifying Not Allowed CI Actions, Compatible CI Actions, and Not Allowed Operational Transitions that restricts some operations and enables for others.

With CI Lifecycle Management you can:

- Manage CI operational states and CI actions throughout the entire CI lifecycle.
- Manage CI operational state transitions.
- Restrict certain operational state transitions.
- Associate certain actions for certain CI types that are in specific operational state.
- Restrict IT Service Management applications based on CI operational state.
- Audit CI operational states and CI actions during the entire CI lifecycle.

Lifecycle management APIs

CI Lifecycle Management provides a set of APIs to manage CI operational state and CI actions during the entire CI lifecycle. All restrictions and allowances specified by rules in the UI are enforced when state management APIs run, and if an API attempts to perform a restricted operation, the operation is blocked and an error is logged.

Registering requestors

When using the lifecycle management APIs to apply CI actions, requestors are required to be registered and to obtain a requestor ID which is unique within the lifecycle management tables. To register and to obtain a requestor ID, non-workflow users should call the registerOperator API. Workflow users can use the active Workflow context as the requestor ID, and they do not need to explicitly call registerOperator.

After completing the CI lifecycle operations, the requestor should call the unregisterOperator API to unregister. All the state management records associated with that specific requestor ID are then marked as inactive or they are removed by the CI Lifecycle Management — Restore Internal State Management Tables scheduled job.

Integration with Incident Management and Problem Management

A base instance includes the pre-defined CI action CreateTask used for creating a task for a CI. New instances have a pre-defined Not Allowed CI Action, specifying that the 'CreateTask' action is not allowed for any CI with a **Retired** operational state. This restriction is integrated with Incident Management and with Problem Management to prevent the creation of incident or problem tasks for retired CIs. The 'CreateTask' CI action is used as a reference qualifier to the Configuration Item field of the Incident/ Problem tables. In a new incident or problem, CIs in which Operational Status is 'Retired' — are filtered out from the Configuration Item list on the form. For more information about reference qualifiers, see Reference qualifiers .

Integration with Asset Management

In a base system, a CI's Operational Status field and the Status/Hardware Status (if its hardware) fields are kept synchronized if one of the two fields' values is **Retired**. When Operational Status of a CI is set to **Retired**, then the Status/Hardware Status field is automatically set to **Retired**. In the opposite direction, when the Status/Hardware Status field of a CI is set to **Retired**, Operational Status is automatically set to **Retired** too.

- When an Operational Status field changes from **Retired** to another status, the CI's Status/Hardware Status field is set to **Installed**.
- When a CI's Status/Hardware Status field changes from **Retired** to another status, the Operational Status field is automatically set to **Non-Operational**.

  The change of state from 'Retired' to another state is seldom, and by default, the state is changed to 'Non-Operational'. However, this might not be the intended state for the record. Therefore, it important that administrators review and manage the state appropriately in this case.

Whenever CI's Status/Hardware Status changes, it is synchronized to the CI's corresponding Asset State field, and vice versa — keeping the CI's Operational Status and the CI's corresponding Asset State synchronized.

For more information about mapping Asset State and Substate fields to a CI's Status/Hardware Status (if its hardware) field, see Map asset state and CI hardware status. And for more information about retiring assets, see Retire assets.

- Get started with CI Lifecycle Management

  Follow these high level steps to get started and to track activities of the CI Lifecycle Management module of the CMDB application.

- Lifecycle management APIs

  CI Lifecycle Management provides a set of state management APIs for manipulating CI operational states, and applying CI actions.

- Components installed by CI Lifecycle Management

  Several types of components are installed by CI Lifecycle Management (included in the com.snc.cmdb plugin), including tables, scheduled jobs, and properties.

- Activate the CI Lifecycle Management scheduled job

  When starting to use the CI Lifecycle Management module, ensure to activate the CI Lifecycle Management - Restore Internal State Management Tables scheduled job which is disabled by default. This scheduled job continuously checks and maintains the data integrity of all internal CI Lifecycle Management tables.

- Define a CI action

  Define a CI Lifecycle Management CI action that can be later applied to CIs.

- Define compatible CI actions

  Allow a CMDB CI Lifecycle Management operation in which two specified CI actions can be applied simultaneously to a CI.

- Define a not-allowed CI action

Define a restriction for CI Lifecycle Management in which a specified action is not allowed for a CI that is in a specified operational state.

- Set priority for an operational state

  CI Lifecycle Management allows multiple operators or automations to simultaneously set different operational states for a CI. A CI can have only a single operational state, so in this case, the cumulative operational state of the CI is set to the one with the highest priority. It is recommended that you specify a priority for each operational state that you define so that a cumulative state can be correctly calculated.

- Define a non-allowed operational transition

  Define a restriction for CI Lifecycle Management in which a specified CI cannot transition from one operational state to another.

Get started with CI Lifecycle Management

Follow these high level steps to get started and to track activities of the CI Lifecycle Management module of the CMDB application.

Before you begin

Role required: none

Procedure

1. Activate the base system CI Lifecycle Management - Restore Internal State Management Tables scheduled job that continuously checks and maintains data integrity of all internal CI Lifecycle Management tables.
2. Define CI actions.
3. Define compatible CI actions rules.
   Navigate to **All > Configuration > CI Lifecycle Management > CMDB CI Actions** to display currently active/inactive CI actions in the CMDB.
4. Define not-allowed CI actions rules.
5. Define not-allowed operational state transitions rules.

6. Define new operational states by modifying the operational_status field in the [cmdb_ci] table in the system dictionary.
   Navigate to **All > Configuration > CI Lifecycle Management > View Internal Operational States** to display available operational states set by each requestor.

7. Set priority for operational states.

8. Call APIs to apply CI actions.
   Navigate to **All > Configuration > CI Lifecycle Management > CMDB CI Actions** to display which actions were submitted and thier active/ inactive state in the CMDB.

9. Navigate to **All > Configuration > CI Lifecycle Management > View CI State Registered Users** to display currently registered operators that were registered via the registerOperator API.

10. Review Renew Lease tasks and extend leases as needed: Navigate to **All > Configuration > CI Lifecycle Management > Renew Lease Tasks**. These tasks are created automatically by the CI Lifecycle Management - Restore Internal State Management Tables scheduled job for CI action records in which the lease for a valid requester has expired. The Requestor should use the lifecycle management API ExtendCIActionLease to extend the lease. Otherwise, if the lease remains expired for a specified grace period, the CI Lifecycle Management - Restore Internal State Management Tables scheduled job marks the respective CI action record as 'inactive'.
    The grace period for expired lease time is configurable by the system property glide.cmdb.statemgmt.max_lease_expired_days.

11. Navigate to **All > Configuration > CI Lifecycle Management > State Management Logs** to display logs of CI Lifecycle Management operations.

Lifecycle management APIs

CI Lifecycle Management provides a set of state management APIs for manipulating CI operational states, and applying CI actions.

State management APIs adhere to restrictions and allowances specified by Not Allowed CI Actions, Compatible CI Actions, and Not Allowed Operational Transitions. If an API attempts to perform a restricted operation, the operation is blocked, an error is logged, and a task is automatically created if appropriate.

Lifecycle management APIs can set operational states and CI actions to CMDB groups by utilizing lifecycle management bulk APIs.

Registration APIs

- registerOperator() - Method to register operator with state management for non-workflow user.
- unregisterOperator(String requestorId) - Method to unregister operator for non-workflow users.
- isValidRequestor(String requestorId) - Method to determine if the specified requestor is a valid active workflow user or a registered user.
- isLeaseExpired(String requestorId, String ciSysId, String ciActionName) - Method to check if registered user lease expired.
- extendCIActionLease(String requestorId, String ciSysId, String ciActionName, String leaseTime) - Method to extend CI Action Lease time, for registered users. If previous lease already expired, extend lease from now.

Operational State APIs

- setBulkCIOperationalState(String requestorId, String sysIdList, String opsLabel, String opsStateListOld) - Method to set Operational State for an array of CIs.
- getOperationalState(String ciSysId) - Method to get CI Operational State.

CI Actions APIs

- addBulkCIAction(String requestorId, String sysIdList, String ciActionName, String ciActionListOld, String leaseTime) - Method to add CI Action for an array of CIs.
- removeBulkCIAction(String requestorId, String sysIdList, String ciActionName) - Method to remove a CI Action for a list of CIs.
- getCIActions(String ciSysId) - Method to get CI Actions.

Not Allowed Action Based on Operational State API

isNotAllowedAction (String ciType, String opsLabel, String actionName) - Method to check if a specific CI action is not allowed for specific Operational State on a CI Type.

Not Allowed Operational State Transition API

isNotAllowedOpsTransition(String ciType, String opsLabel, String transitionOpsLabel) - Method to check if specific operational state transition is not allowed on a CI Type.

Compatible Action API

isCompatibleCIAction(String actionName, String otherActionName)- Method to check if two specific actions are compatible with each other.

Example: Using state management APIs

```
// 1. Register Operator with State Mgmt
var output = SNC.StateManagementScriptableApi.registerOpe
rator();
var jsonUntil = new JSON();
var result = jsonUntil.decode(output);
var requestorId = result.requestorId;

// Get list of sys_ids to update
var sys_ids;

// 2. Set list of sys_ids's Operational State to 'Repair
in Progress'
output = SNC.StateManagementScriptableApi.setBulkCIOperat
ionalState(requestorId, sys_ids,'Repair in Progress');
gs.print(output);

// 3. Set list of sys_ids's CI Action State to 'Patching'
output = SNC.StateManagementScriptableApi.addBulkCIAction
(requestorId, sys_ids, 'Patching');
gs.print(output);
```

Components installed by CI Lifecycle Management

Several types of components are installed by CI Lifecycle Management (included in the com.snc.cmdb plugin), including tables, scheduled jobs, and properties.

> **Note:** The Application Files table lists the components that are installed with this application. For instructions on how to access this table, see Find components installed with an application.

Scheduled jobs installed

Scheduled job	Description
CI Lifecycle Management - Restore Internal State Management Tables	Continuously checks and maintains the data integrity of all internal CI Lifecycle Management tables.
Update life cycle from legacy	Updates CIs Life cycle stage and Life cycle status fields when legacy status fields change.
Update legacy from CSDM"	Updates CIs legacy status fields when life cycle changes.

Tables installed

Table	Description
CI State Registered Users [statemgmt_register_users]	All currently active registered users that were created via the registerOperator API. You cannot manually add new records to this table.
CI Actions	A set of CI actions that can be applied to a CI during its lifetime.

Table	Description
[statemgmt_ci_actions]	
CMDB CI Actions [statemgmt_cmdb_actions]	Active/inactive CI actions set by a specific requestor for a specific CI. You cannot manually add new records to this table.
Compatible CI Actions [statemgmt_compat_actions]	Set of rules that define pairs of CI actions that are compatible for a CI and can be applied simultaneously.
Not Allowed CI Actions [statemgmt_not_allow_actions]	Set of rules that define specific actions that are not allowed for a CI when its in a specific operational state.
Internal Operational States [statemgmt_ops_state]	Internal operational states set by a specific active requestor for a specific CI. You cannot manually add new records to this table.
Renew Lease Task [statemgmt_renew_lease_task]	Set of tasks that were automatically created to renew the lease of CI actions whose lease has expired. You cannot manually add new records to this table.
Operational State Priorities [statemgmt_ops_state_pri]	Priorities of operational states which determine precedence when multiple operational states are set for same CIs by different requestors.
Not Allowed Operational Transitions [statemgmt_not_allow_ops]	Set of rules that define specific operational state transitions that are not allowed.

Properties installed

Property	Description
glide.cmdb.statemgmt.max_lease_expired_days	Maximum number of days that lease expiration can be set with for CI Actions. • Type: integer • Default value: 15 • Location: System Property [sys_properties] table.
glide.cmdb.statemgmt.max_bulk_count	Maximum number of CIs that CI Lifecycle Management can process in a bulk update operation. • Type: integer • Default value: 1000 • Location: System Property [sys_properties] table.

Activate the CI Lifecycle Management scheduled job

When starting to use the CI Lifecycle Management module, ensure to activate the CI Lifecycle Management - Restore Internal State Management Tables scheduled job which is disabled by default. This scheduled job continuously checks and maintains the data integrity of all internal CI Lifecycle Management tables.

Before you begin

Role required: none

About this task

When CI Lifecycle Management operations do not complete properly, for example due to a failure of the requestor or a requestor whose lease has expired, the integrity of tables related to CI Lifecycle Management might be compromised. The CI Lifecycle Management - Restore Internal State Management Tables scheduled job scans tables related to CI Lifecycle Management, and does the following:

- De-activates or removes all internal lifecycle management records with invalid requestors, and closes any corresponding Renew Lease Tasks if present.
- Detects records associated with a valid requestor whose lease has expired, and automatically creates a Renew Lease Task to notify the user and to provide details for extending the lease. If the requestor takes no action and the lease remains expired for a specified grace period (default 15 days), automatically de-activates the corresponding CI action record, and closes any corresponding Renew Lease Task if present.

Procedure

1. Navigate to **System Definition**, and click **Scheduled Jobs**.
2. Search for the CI Lifecycle Management - Restore Internal State Management Tables job.
3. In the respective **Active** column, double-click the value false, and select true.
4. Click the **Save** icon.

Define a CI action

Define a CI Lifecycle Management CI action that can be later applied to CIs.

Before you begin

Role required: none

About this task

You can view a list of all the actions that are currently applied to CIs by navigating to **Configuration** and clicking **CMDB CI Actions**.

Procedure

1. Navigate to **All > Configuration > CI Lifecycle Management > CI Actions**.
2. On the CI Actions page, select **New**.
3. Enter **Name** and **Description**.
4. Select **Submit**.

Define compatible CI actions

Allow a CMDB CI Lifecycle Management operation in which two specified CI actions can be applied simultaneously to a CI.

Before you begin

Role required: none

About this task

By default, it is not allowed to apply more than a single action to a CI. You can change that behavior by defining pairs of CI actions as compatible and therefore these actions can be applied simultaneously to a CI. For example you can specify that Provisioning and Patching are compatible CI actions, which lets you apply both to a CI at the same time.

Procedure

1. Navigate to **All > Configuration > Compatible CI Actions**.
2. On the **Compatible CI Actions** page click **New** and fill out the form.

Compatible CI Actions

Field	Description
Action	First action in the compatibility actions pair.
Compatible Action	Second action in the compatibility actions pair.

Result

An API can successfully apply the two specified actions simultaneously to a CI.

Define a not-allowed CI action

Define a restriction for CI Lifecycle Management in which a specified action is not allowed for a CI that is in a specified operational state.

Before you begin

Role required: none

About this task

By default, there are no restrictions in the CMDB CI Lifecycle Management on applying CI actions. You can restrict this behavior by not allowing a specified action to be applied to a CI when it is in a specified operational state. For example, you can define a restriction in which the provisioning action cannot be applied to a Linux Server that is in a non-operational state.

Procedure

1. Navigate to **All > Configuration > CI Lifecycle Management > Not Allowed CI Actions**.
2. Click **New** on the **Not Allowed CI Actions** page, and fill out the form.

Field	Description
Not Allowed Action	The action that is being restricted.
CI Type	The CI type for which the restriction applies to. To apply a rule to all CIs, select Configuration Item.
Operational State	The operational state that the CI must be at in order to apply the restriction.

3. Click **Submit**.

Result

If an API attempts to apply the specified action to the specified CIs, while it is in the specified operational state, the operation fails and an error is logged.

Set priority for an operational state

CI Lifecycle Management allows multiple operators or automations to simultaneously set different operational states for a CI. A CI can have only a single operational state, so in this case, the cumulative operational state of the CI is set to the one with the highest priority. It is recommended that you specify a priority for each operational state that you define so that a cumulative state can be correctly calculated.

Before you begin

Role required: none

About this task

Procedure

1. Navigate to **All > Configuration > CI Lifecycle Management > Operational State Priority**.

2. On the **Operational State Priority** page, click the operational state for which you want to set or update priority.
3. Enter a **Priority** and click **Update**.
   Smaller numbers represent higher priority.

Define a non-allowed operational transition

Define a restriction for CI Lifecycle Management in which a specified CI cannot transition from one operational state to another.

Before you begin

Role required: none

About this task

By default, CI Lifecycle Management has no restrictions for transitioning CIs from one operational state to another. You can restrict this behavior by defining transitions that are not allowed for a specified CI. For example, you can define a restriction on transitioning a Linux server from non-operational state to repair in progress state.

Procedure

1. Navigate to **All > Configuration > CI Lifecycle Management > Not Allowed Operational Transitions**.
2. On the **Not Allowed Operational Transitions** page, click **New** and fill out the form.

Field	Description
CI Type	The CI type for which the restriction applies.
Not Allowed Transition	The CI state into which transitioning is restricted.
Operational State	The operational state that the CI must be in for the restriction to apply.

Result

If an API attempts to transition a CI that is in the specified operational state to a state that is not allowed, the operation fails and an error is logged.